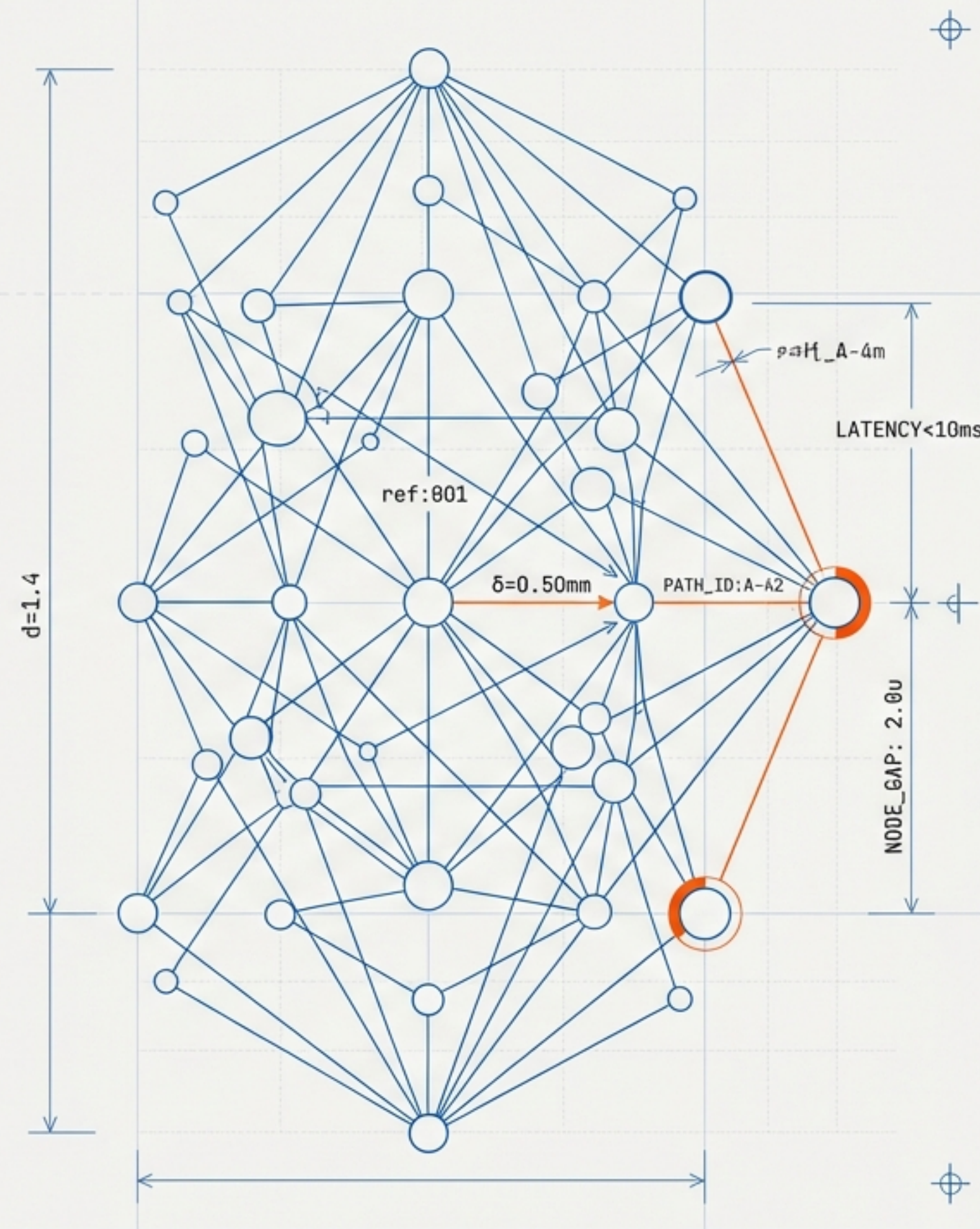


FLeT REST API Implementation

Technical Brief v1.4.50 | Architecture,
Observability, and Orchestration

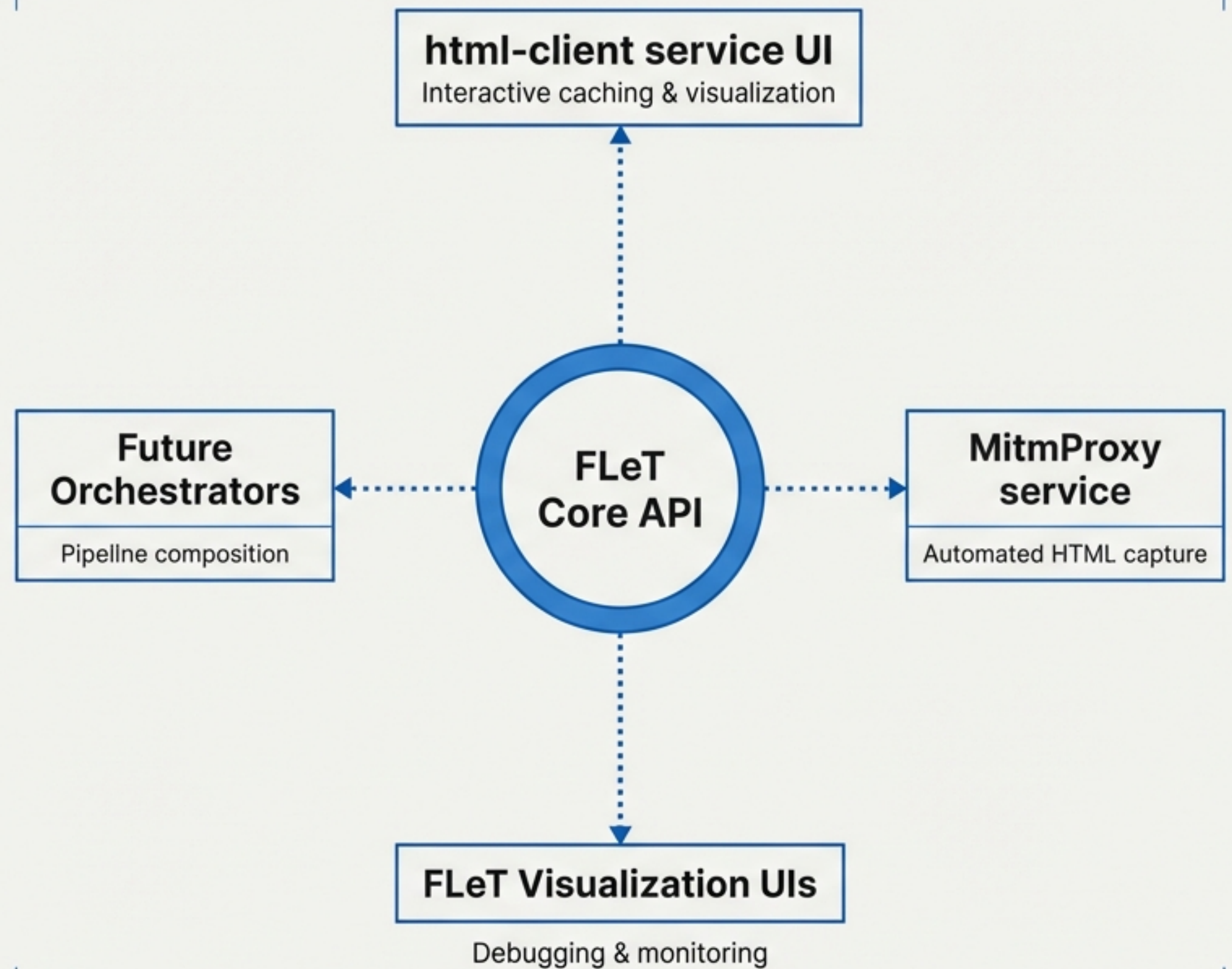
Package: mgraph_ai_service_html_graph | Date: 19 January 2026



EXPOSING FLeT CAPABILITIES TO EXTERNAL CLIENTS

Core Objective: Building a comprehensive REST API layer that exposes Flow-Level Execution Task (FLeT) operations for HTML caching, retrieval, and deep observability.

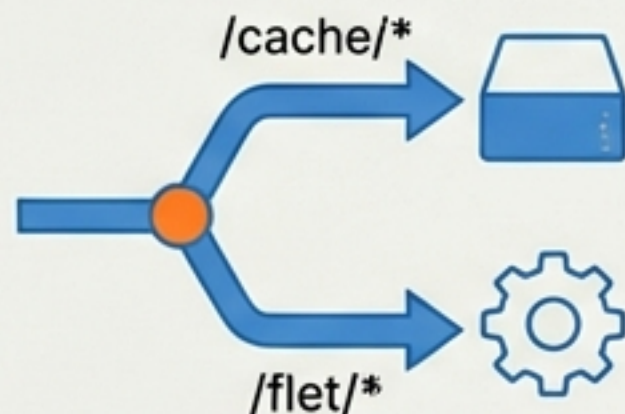
The Value Proposition: Transforming internal debugging tools into external-facing, robust features.



FOUR PILLARS OF ARCHITECTURAL DESIGN

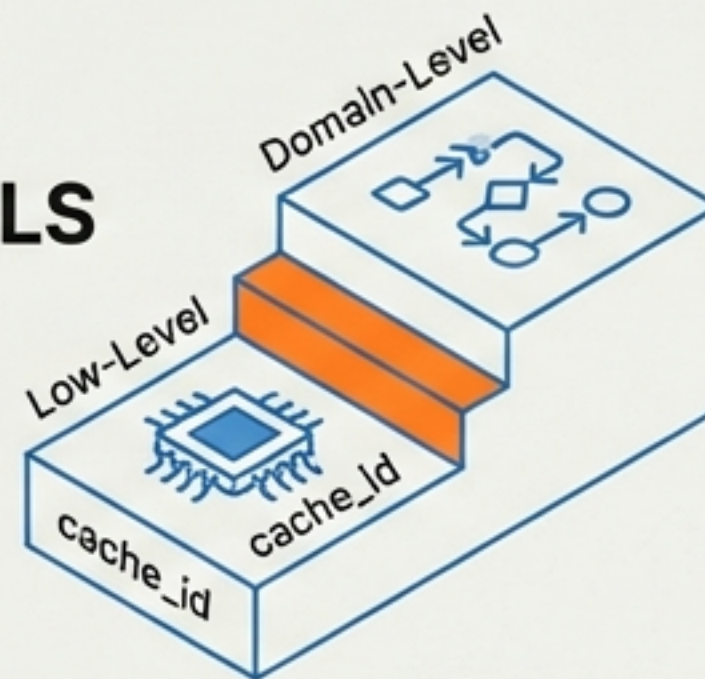
SEPARATION OF CONCERNS

Strict namespace separation.
'/cache/*' handles data storage (migratable).
'/flet/*' handles execution logic.



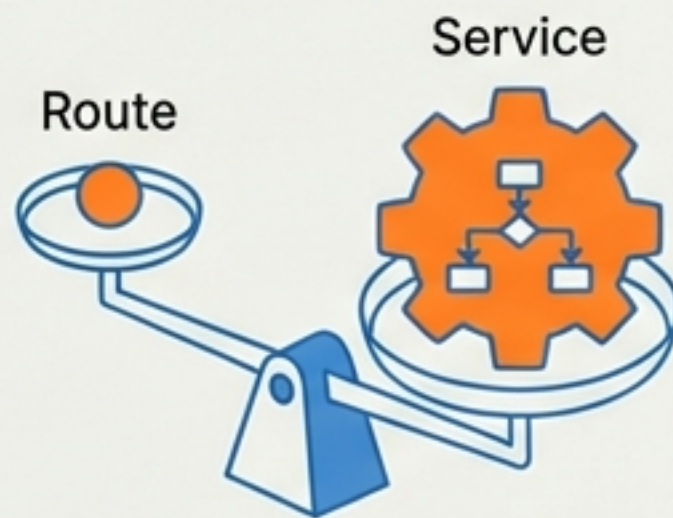
MULTIPLE ABSTRACTION LEVELS

Low-Level: Explicit 'cache_id' control.
Domain-Level: Workflow orchestration.



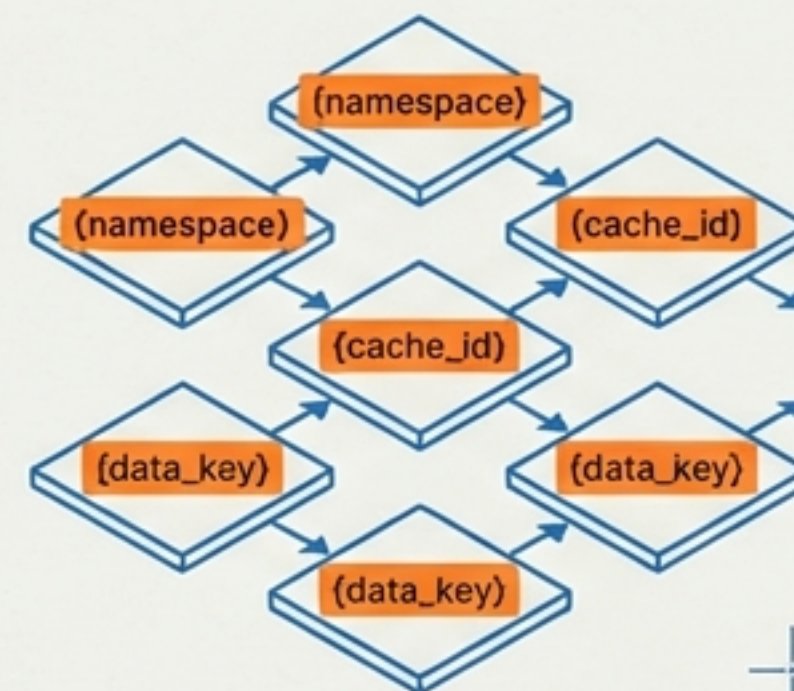
THIN ROUTES, RICH SERVICES

Routes: Parse & Format.
Services: Business Logic & Orchestration.



CONSISTENT PATTERNS

Predictable URL structures: '{namespace}', '{cache_id}', '{data_key}'.



HIGH-LEVEL SYSTEM ARCHITECTURE

External Layer

Clients (UI, MitmProxy, Orchestrators)

Interface Layer

REST API Router (FastAPI)

Logic Layer

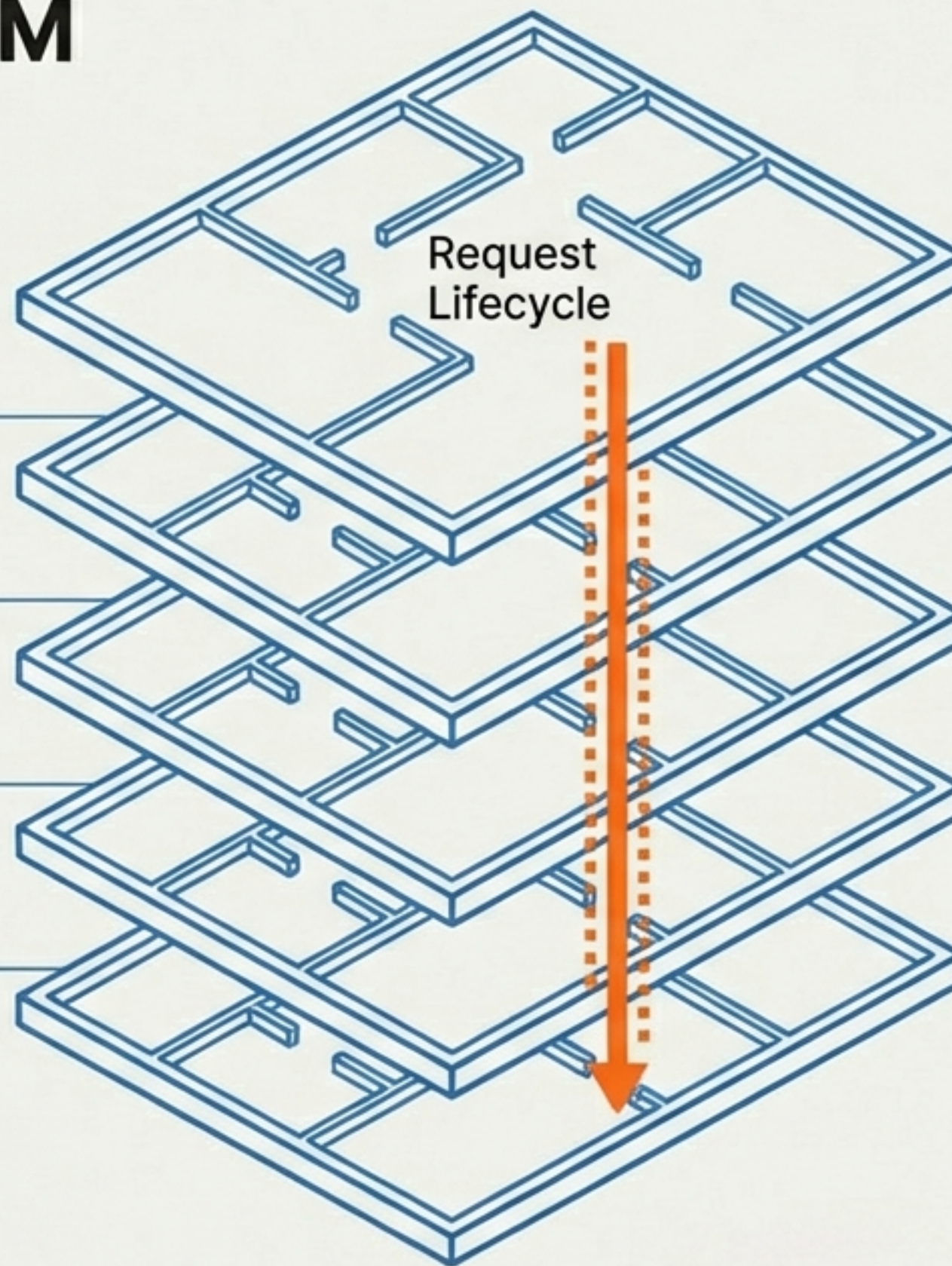
Service Classes (Rich Logic)

Execution Layer

FLeT Pipeline (Flows, Tasks)

Storage Layer

Cache Storage (Entities & Data Files)



↓ **Down:** Request Flow
↑ **Up:** Observability Data

THE FOUNDATION: CACHE ENTITY ROUTES

Namespace: `/cache/{namespace}/entity/`

These routes manage the lifecycle of cache entries.

They are the 'Nouns'.
They are the 'Nouns' of the system.

Designed to be migratable to a dedicated Cache Service in the future.

Route	Method	Purpose
<code>/create</code>	[POST]	Create new entity from cache_key
<code>/lookup</code>	[POST]	Find entity by key or hash
<code>/cache_id</code>	[GET]	Retrieve full entity data
<code>/cache_id/metadata</code>	[GET]	Retrieve only metadata
<code>/cache_id/refs</code>	[GET]	Retrieve only references
<code>/cache_id/exists</code>	[GET]	Lifecycle check

DATA MANAGEMENT: CACHE DATA ROUTES

Namespace: /cache/{namespace}/data/{cache_id}/

Purpose: Direct file system interactions for hierarchical data paths.

Navigation

- [GET] / (List files, recursive support)
- [GET] /paths (List distinct paths)

Content Access (Typed)

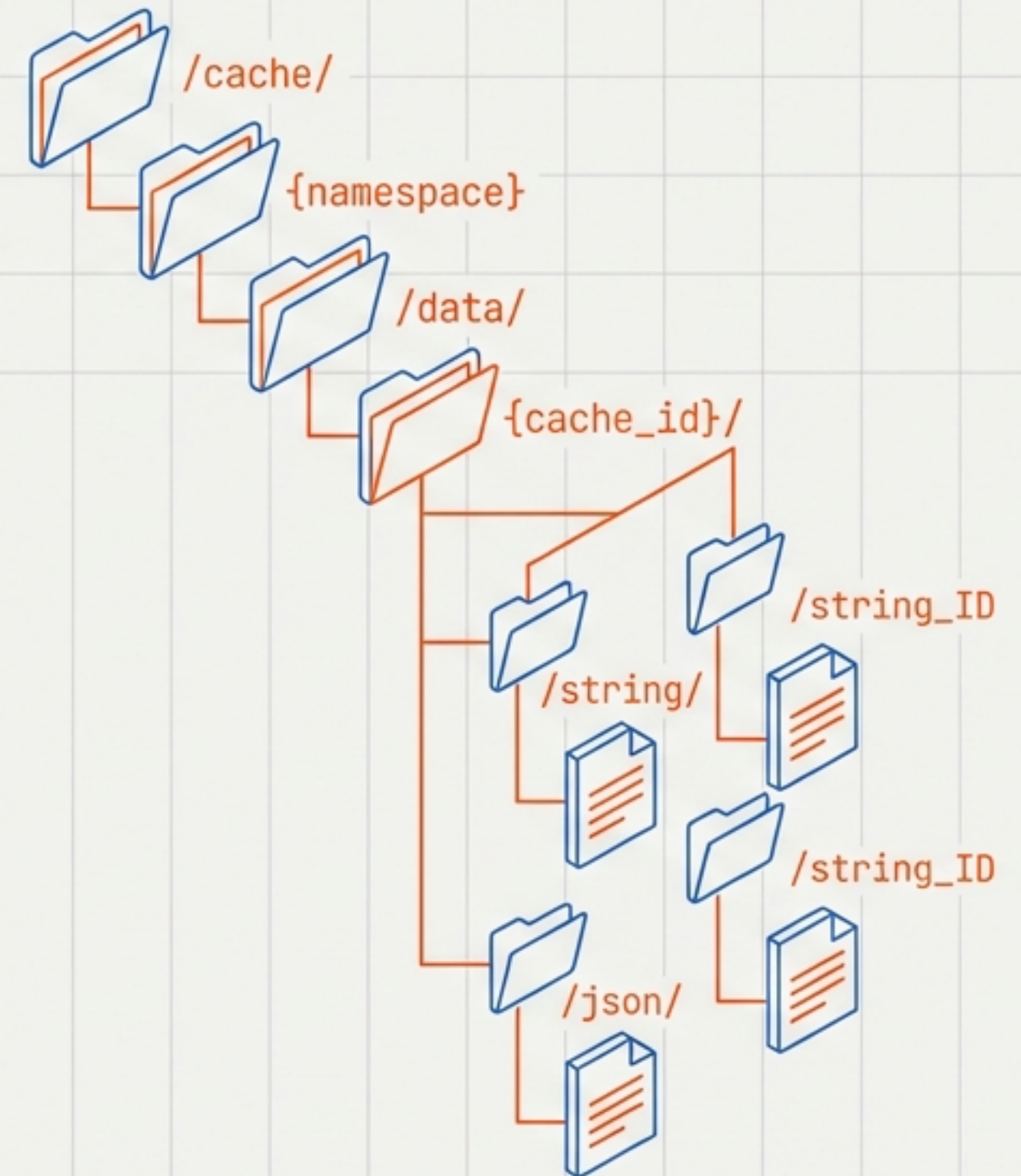
- [GET] /string/{data_key:path}/{data_file_id}
- [GET] /json/{data_key:path}/{data_file_id}

Modification

- [POST] /store/{type}/... (Create new)
- [PUT] /{type}/... (Update existing)

Verification

- [GET] /exists/{data_type}/...



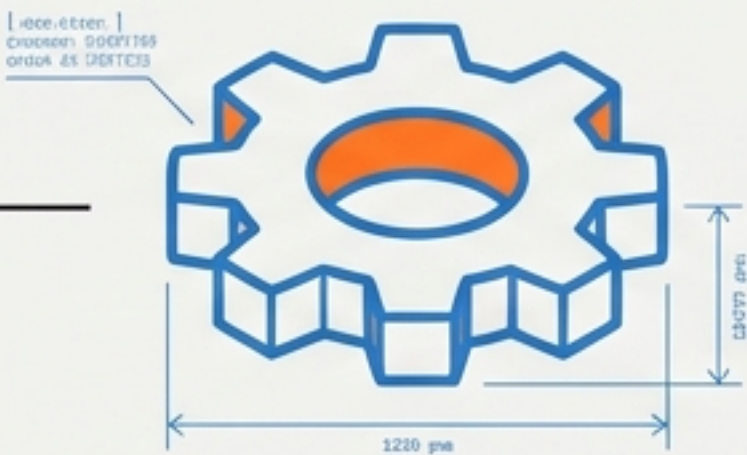
THE ENGINE: FLeT EXECUTION ROUTES

MANUAL MODE

TIER 1: LOW-LEVEL CONTROL

ROUTE: `/flet/html/to/cache/{namespace}/{cache_id}`

For Power Users. Requires pre-existing cache_id. Explicit input control.



AUTOMATIC MODE

TIER 2: DOMAIN-LEVEL ORCHESTRATION

ROUTE A: `/flet/html/store/{namespace}/url`

Convenience Workflow. Service handles entity creation, key generation, and flow execution.

ROUTE B: `/flet/html/load/{namespace}/id`

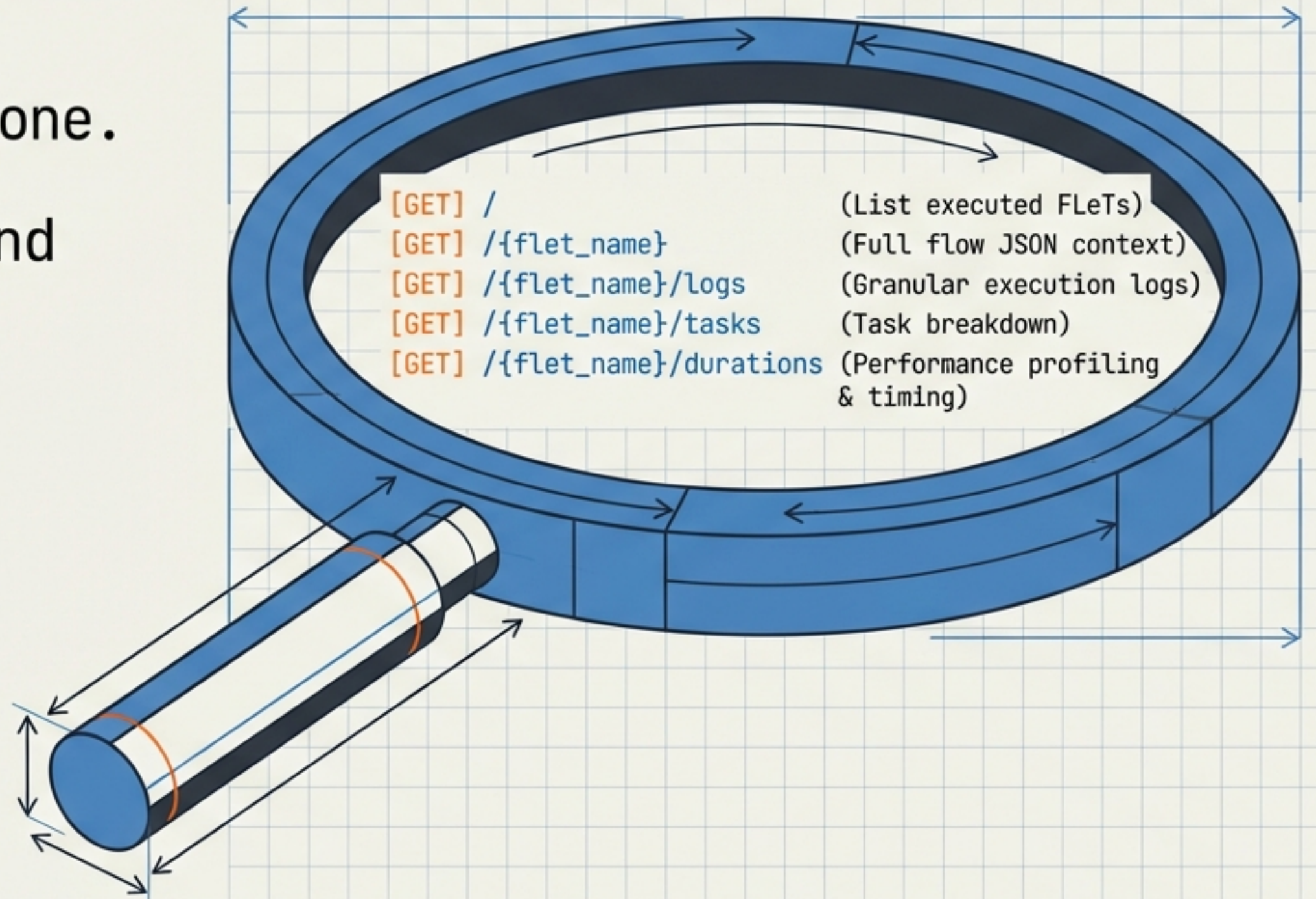
Smart retrieval using identifiers.



THE LENS: FLOW OBSERVABILITY

Namespace: `/flet/flows/{namespace}/{cache_id}/`

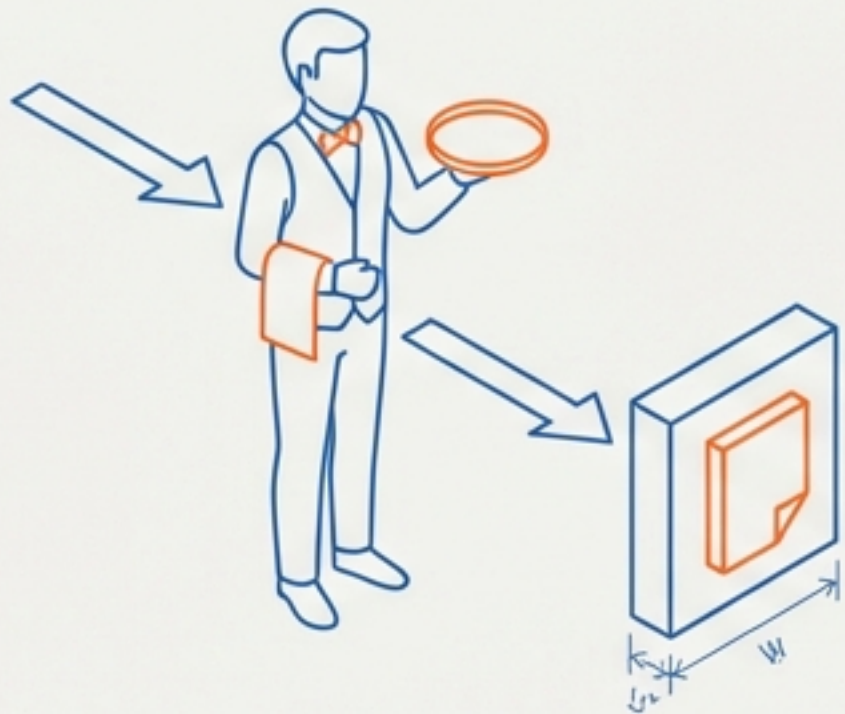
Unlike standard APIs, FLeT exposes HOW the work was done. This enables debugging, performance monitoring, and visualization UIs.



DESIGN PATTERN: THIN ROUTES VS. RICH SERVICES

The Route Class (Interface)

The Waiter



- Request Parsing
- Service Calls
- Response Formatting
- HTTP Error Handling

Routes__Cache__Entity

The Service Class (Brains)

The Chef

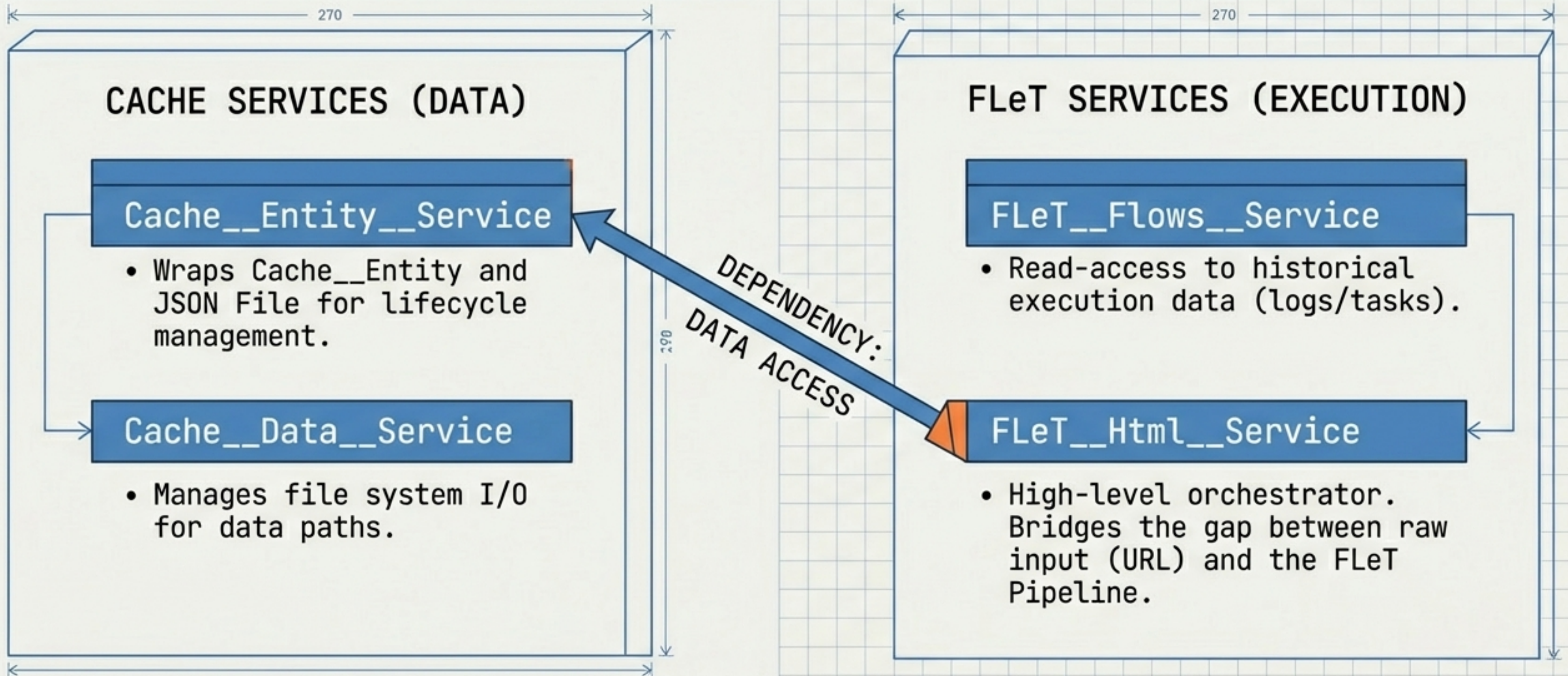


- Business Logic
- Entity Management
- FLeT Orchestration
- Complex Validations

Cache__Entity__Service

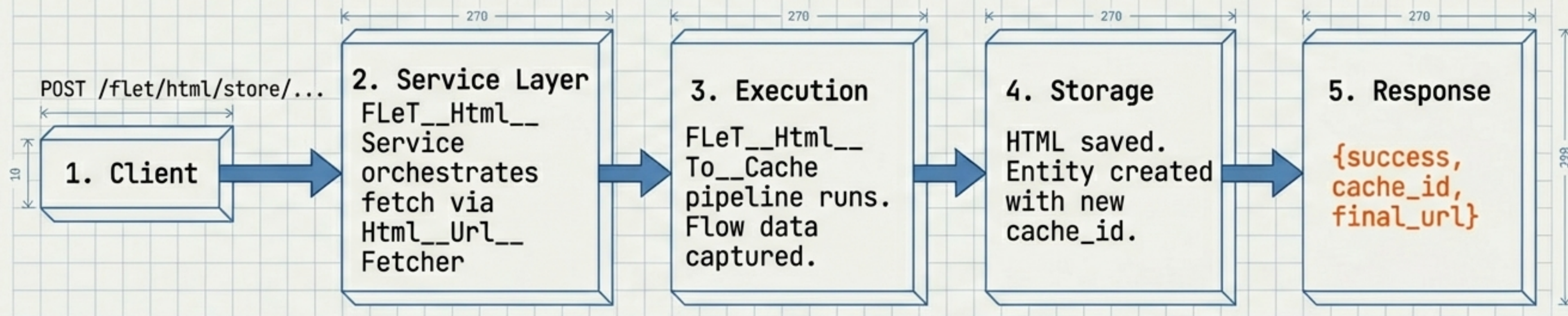
SERVICE LAYER BREAKDOWN

ENGINEERING SPECIFICATION: SERVICE ARCHITECTURE & DEPENDENCIES



DATA FLOW SCENARIO: STORE HTML FROM URL

ENGINEERING SPECIFICATION: SEQUENCE OF OPERATIONS FOR DATA INGESTION



GAP ANALYSIS: LEVERAGING VS. BUILDING

ENGINEERING SPECIFICATION: COMPONENT ASSESSMENT & REQUIREMENT DEFINITION

EXISTING COMPONENTS (LEVERAGED)

- ✓ Pipelines: `FLeT__Html__To__Cache`
- ✓ Wrappers: `Html_Cache__Client`
- ✓ Utilities: `Html__Url__Fetcher`
- ✓ Base: `Html_FLeT__Base`

NEW IMPLEMENTATION (REQUIRED)

- ✗ Service Classes (Business Logic)
- ✗ Route Classes (FastAPI Endpoints)
- ✗ Schemas (Pydantic DTOs)

IMPLEMENTATION PHASES 1 & 2

ENGINEERING SPECIFICATION: DEVELOPMENT TIMELINE & DELIVERABLES

PHASE 1: CACHE ROUTES (THE FOUNDATION)

Action Items

- Implement Cache__Entity__Service & Routes
- Implement Cache__Data__Service & Routes

Deliverable

Full CRUD on entities and data files.

PHASE 2: FLOW OBSERVABILITY (THE EYES)

Action Items

- Implement FLeT__Flows__Service
- Expose logs, tasks, and durations via API

Deliverable

Visibility into the "Black Box" of execution.

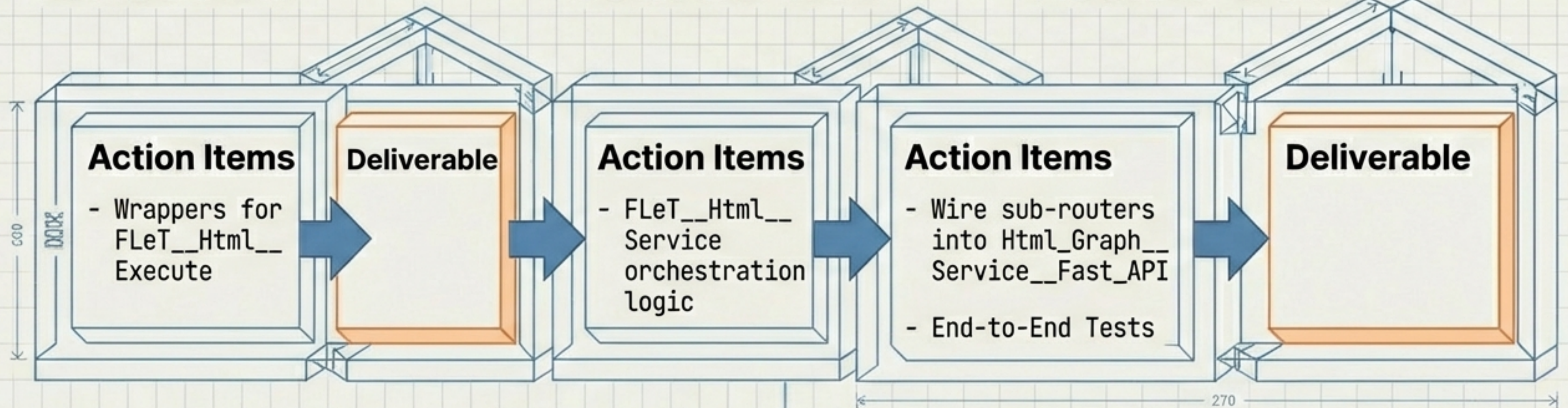
IMPLEMENTATION PHASES 3, 4, & 5

ENGINEERING SPECIFICATION: DEVELOPMENT TIMELINE & DELIVERABLES

PHASE 3: LOW-LEVEL EXECUTION

PHASE 4: DOMAIN-CONVENIENCE

PHASE 5: MAIN AGGREGATOR



TESTING STRATEGY

Unit tests alongside implementations. Mocked service calls for Route tests.
Integration tests for full workflows.

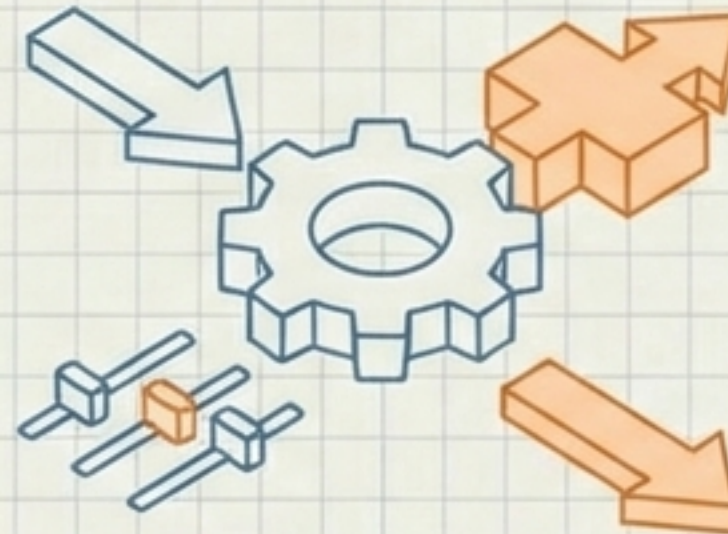
STRATEGIC IMPACT & SUMMARY

ENGINEERING SPECIFICATION: SYSTEM ARCHITECTURE & DEVELOPMENT CONCLUSION



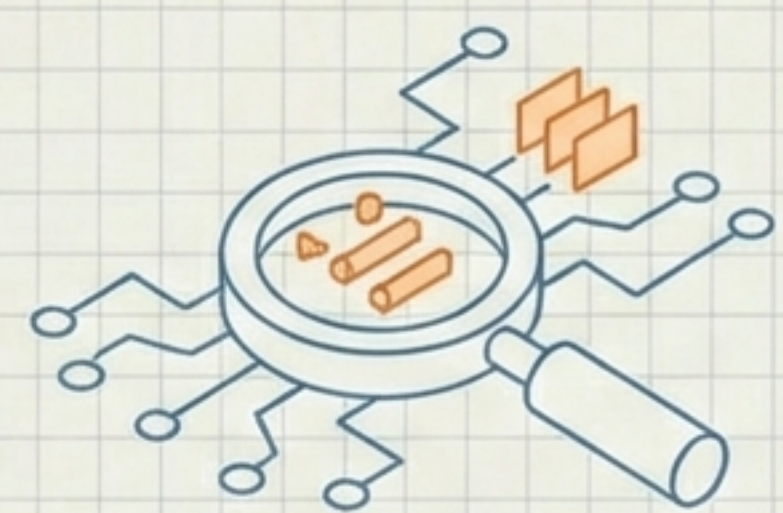
DECOUPLING

Complete separation of Cache
vs. Execution operations.
Future-proofed for
microservice migration.



FLEXIBILITY

Supports both granular
control (Devs) and high-level
workflows (Apps).



TRANSPARENCY

Rich observability
endpoints turn debugging
into a feature.

STATUS: SPECIFICATIONS DEFINED. READY FOR PHASE 1 IMPLEMENTATION.